

We'll learn a few algo designing techniques.

- Divide & Conquer
- Greedy Tech.
- Dynamic Prog algo.

Analysis
of
algo

Algorithm — (1) Correctness of the algo. (Mathematical)

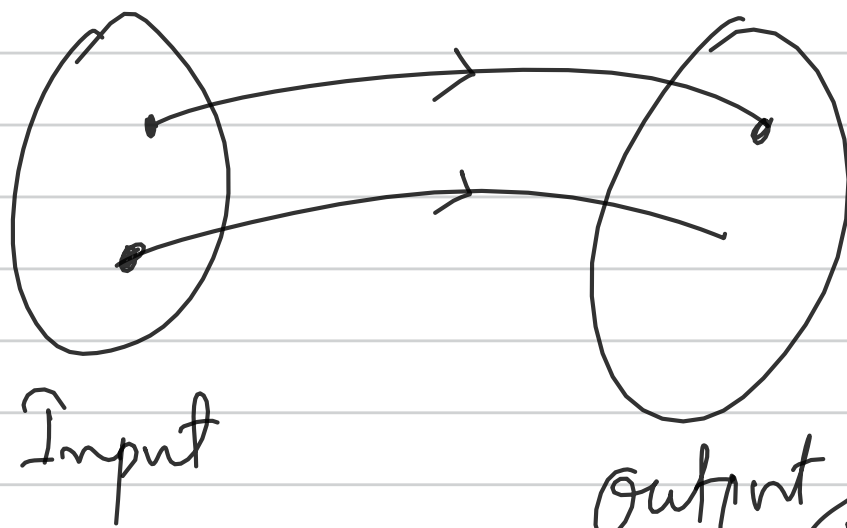
(2) Efficient algo.

Time Complexity ← (how fast)

Space Complexity ← (how much memory used)

Class P // Class NP // Class NP-Complete.

Computational Prob



Eg: Sorting Prob.

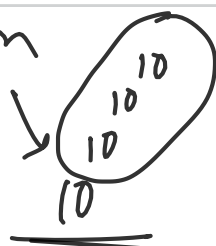
Input size = (n)

Input: A set of n nos $a_1, a_2, \dots, a_n$

Output: Those nos in inc/dec order.

$$\sum_{i=1}^n \log_2 a_i$$

$7 \rightarrow 111$



Algorithm

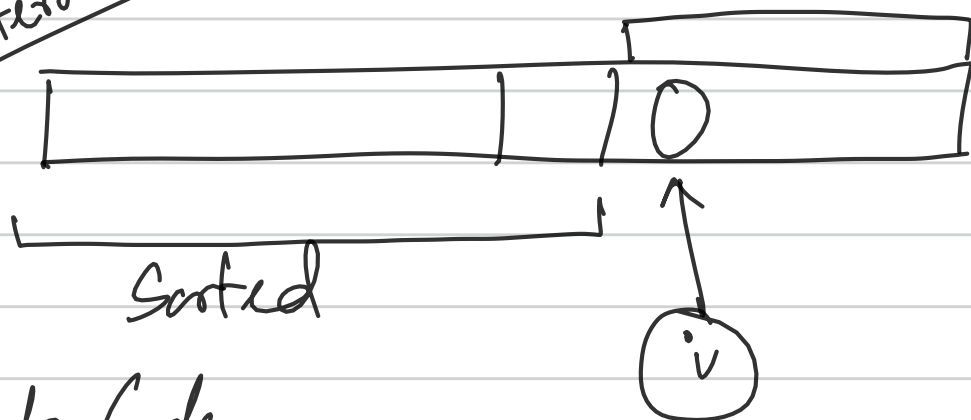
Its a set of well defined rules for solving some computational problems.

Insertion Sort.

[3 | 9 | 11 | 21 | 12 | 16 | 15]

9 11 3 21
key = 11

After i-th iteration



Selection Sort
[3 9 11 12] $\Rightarrow$

Pseudo-Code.

1. for $i=1$ to $n-1$

2. $key = A[i]$

3. $j = i-1$

4. while ($A[j] > key$) and ($j \geq 0$) do

5. $A[j+1] = A[j]$

6. $j = j-1$

7. $A[j+1] = key.$

$C_1 n$

$C_2 (n-1)$

$C_3 (n-1)$

$C_4 \sum_{i=1}^{n-1} t_i$

$C_5 \sum_{i=1}^{n-1} (t_i - 1)$

$C_6 \sum_{i=1}^{n-1} (t_i - 1)$

$C_7 (n-1)$

t_i = no of times the conditions of while loop is being checked for i th element

Selection Sort



Correctness of Insertion Sort:

By induction:

Base Case: $t = 1$. (True)

Ind hyp: After $(t-1)$ iterations, first $(t-1)$ nos of the input array is sorted.

Ind Step: While inserting the last no, we find its correct position between a & b
 $a \leq x$ and $b \geq x$

→ Unique.

(assume all the elements are distinct)

⊗ Finished t iterations

and $\Rightarrow$ first t elements of the input array is sorted.

Efficiency of Algo:

Many diff implementations

Write the code

A_1

A_2

C_1

C_2

Infinite inputs.

// Run this for every possible inputs

$I_1, I_2, \dots$

$I_1, I_2, \dots$

$\max \{t_1, t_2, \dots\}$

$\max \{t'_1, t'_2, \dots\}$

system dependent // system time

~~min~~

~~min~~

min

(*) We calculate no of primitive operations performed in terms of the input size.

Defn Input size: no of bits required to represent the input.

arithmetic $+, -, /, *$
 logical $>, <, =$
 assignments $\text{key} = A[i]$

Total no of Primitive Ops =

$$T(n) = C_1 n + C_2(n-1) + C_3(n-1) + C_4 \sum_{i=1}^{n-1} t_i + C_5 \sum_{i=1}^{n-1} (t_i - 1) + C_6 \sum_{i=1}^{n-1} (t_i - 1) + C_7(n-1)$$

(*) Neglect — lower order terms.
— constant coefficients.

Best Case: $t_i = 1 \forall i$ ($\Rightarrow$ array is sorted already)

$$\begin{aligned}
 T(n) &= C_1 n + C_2(n-1) + C_3(n-1) + C_4(n-1) + C_5 \cdot 0 + C_6 \cdot 0 + C_7(n-1) \\
 &= n(C_1 + C_2 + C_3 + C_4 + C_7) + (-C_2 - C_3 - C_4 - C_7) \\
 &= \underline{\underline{O(n)}} \quad (\text{Big Oh of } n) \quad (\text{Defn})
 \end{aligned}$$

Worst Case: $t_i = i$ for all i ($\Rightarrow$ array is also sorted but in opp order)

$$T(n) = C_1 n + C_2(n-1) + C_3(n-1) + C_4 \sum_{i=1}^{n-1} i + C_5 \sum_{i=1}^{n-1} (i-1) + C_6 \sum_{i=1}^{n-1} (i-1) + C_7(n-1)$$

$$= n^2 (c_1 + c_2 + c_3) + n (c_1 + c_2 + \dots) + (-c_2 - c_3 - \dots)$$

$$= \underline{\underline{O(n^2)}} \quad (\text{Defn later})$$